NEXO IDE 5.0

Intérprete educativo de pseudocódigo

Proyecto: Desarrollo de un intérprete educativo de pseudocódigo

Curso: Autómatas y Lenguajes Formales

Carrera: Ingeniería en Sistemas de Información

Institución: _____

Equipo: _____

Docente: _____

Fecha: octubre de 2026

Entrega: Aplicación web local, código fuente, 44 casos de prueba y evidencias visuales.

1. Introducción

Nexo IDE es una aplicación educativa que permite redactar, analizar y ejecutar algoritmos de pseudocódigo. Su propósito es facilitar la comprensión de variables, tipos de datos, expresiones, estructuras condicionales, clasificación de tokens y reglas gramaticales. La versión 5 añade tres ciclos (Para, Mientras y Repetir) y un espacio de trabajo multiarchivo con pestañas acumulativas, además del diccionario de tipos anterior.

El proyecto aplica expresiones regulares, reconocimiento léxico, gramática independiente de contexto, análisis sintáctico descendente recursivo y evaluación controlada de un árbol sintáctico abstracto. El pseudocódigo no se ejecuta como JavaScript y no se emplea eval().

2. Tecnologías y guía de ejecución

La interfaz utiliza HTML5, CSS3 y JavaScript puro; no necesita conexión a internet ni instalación. Para iniciar, abra index.html en un navegador. La biblioteca lateral ofrece diez programas y el inspector incorpora pestañas para Resumen, Tokens, Variables, Tipos, Ciclos y Sintaxis. Los archivos abiertos se conservan como pestañas independientes, con nuevo archivo, importación múltiple, renombrado, cierre y exportación del archivo activo. Node.js es opcional y solo se necesita para repetir las pruebas automatizadas.

```
node tests/run-tests.js
node tests/run-types-tests.js
node tests/run-loops-tests.js
```

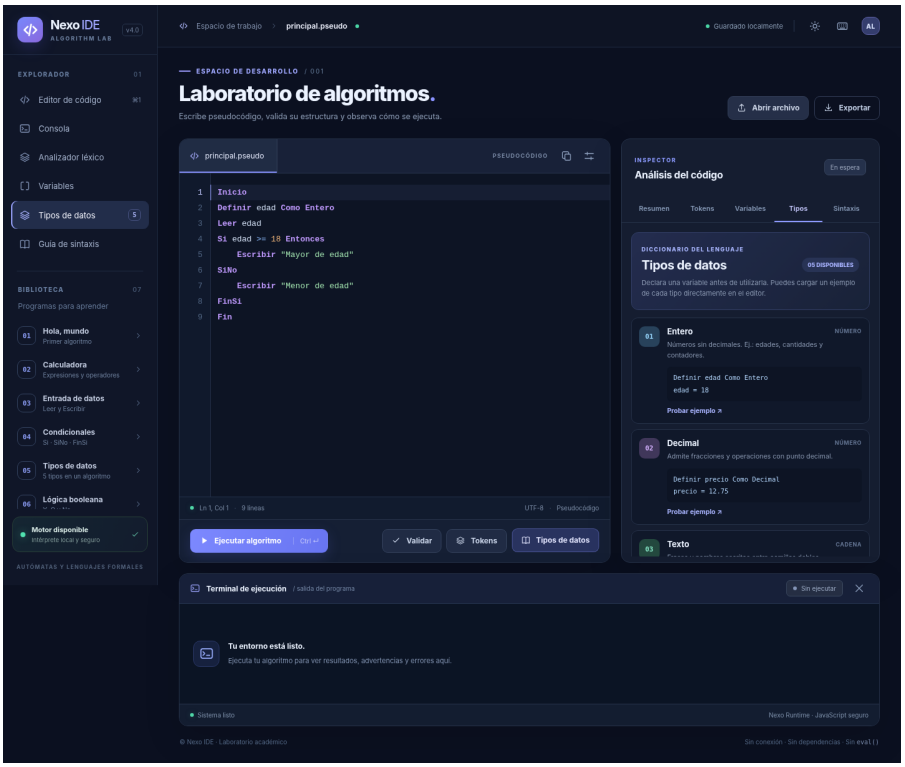
3. Nuevos tipos de datos disponibles

Tipo	Ejemplo	Uso
Entero	18, -4	Números enteros
Decimal (Real)	12.75, 5	Números con o sin decimales
Texto	"Ana"	Cadenas entre comillas dobles
Logico (Booleano)	Verdadero, Falso	Valores booleanos
Caracter	'A', 'Ñ'	Un solo símbolo

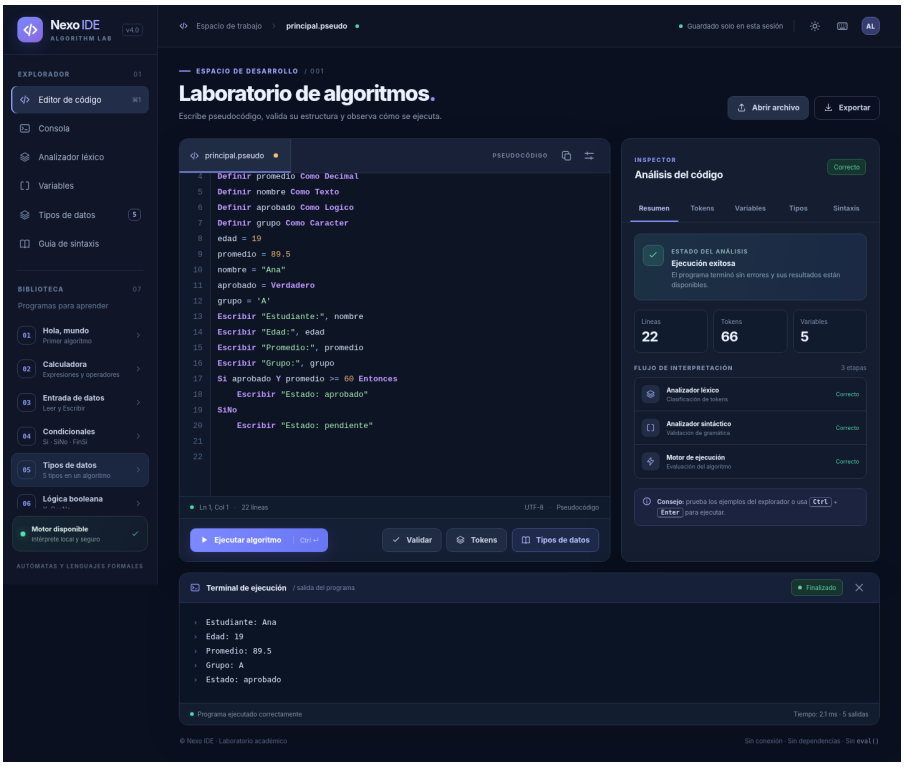
También se aceptan declaraciones múltiples como Definir a, b Como Entero. Para condiciones se incluyen Y, O, No, Verdadero y Falso; Leer convierte los datos al tipo declarado y Escribir puede presentar varias expresiones separadas por comas.

4. Evidencias visuales del panel de tipos

4.1. Panel de tipos de datos con ejemplos cargables



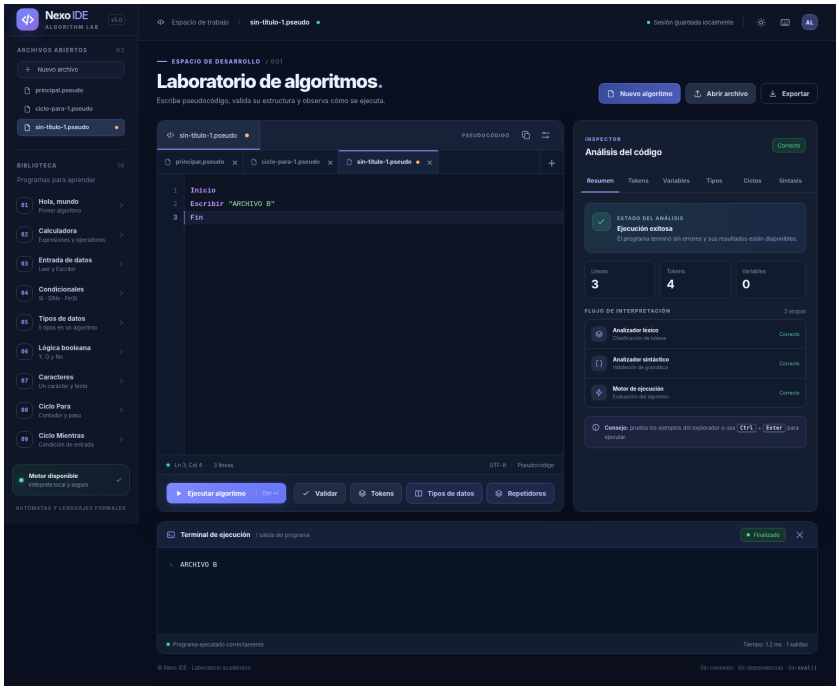
4.2. Ejecución real de un programa con cinco tipos



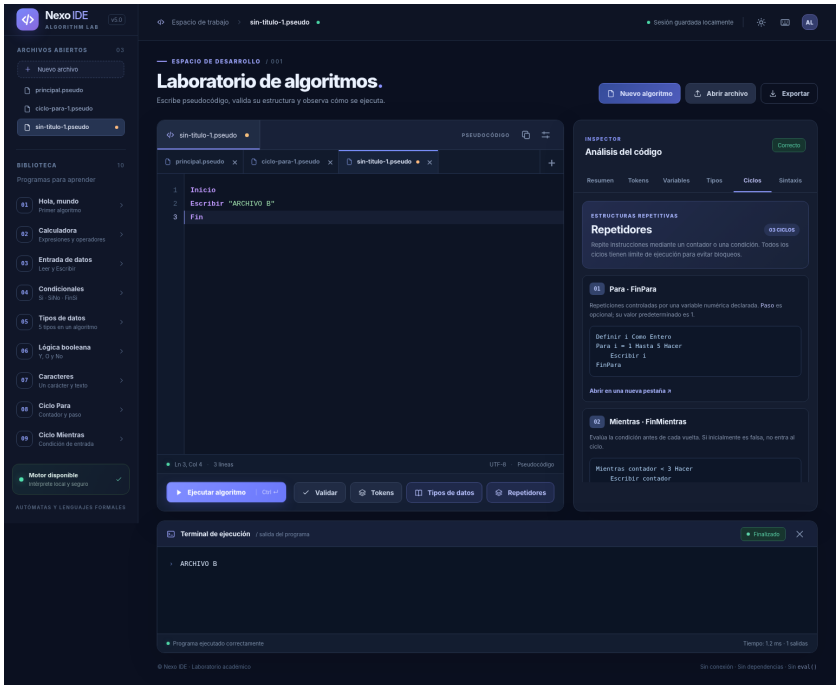
4.3. Adaptación de la interfaz a pantalla móvil



4.4. Espacio de trabajo con varias pestañas de algoritmos



4.5. Inspector integrado de estructuras repetitivas



4.6. Pestañas y ciclos en vista móvil



5. Análisis léxico, gramática y sintaxis

Tokens admitidos: KW (palabras reservadas), ID (identificadores), NUMBER (números), STRING (texto), CHAR (carácter), SYM (símbolos), NL (salto de línea) y EOF (final del archivo). El lexer emplea expresiones regulares. El parser verifica declaraciones, asignaciones, entrada, salida, condicionales, ciclos Para/Mientras/Repetir, literales, paréntesis y precedencia.

```
Programa  -> Inicio NL Bloque Fin
Bloque    -> (Sentencia NL)*
Sentencia -> Definir Ids Como Tipo | ID = Expr | Leer ID
          | Escribir Args | Si Expr Entonces NL Bloque SinoOpt FinSi
```

```
          | Para ID = Expr Hasta Expr PasoOpt Hacer NL Bloque FinPara
          | Mientras Expr Hacer NL Bloque FinMientras
          | Repetir NL Bloque Hasta Que Expr
PasoOpt   -> Paso Expr | epsilon
Ids       -> ID (, ID)*
Tipo      -> Entero | Decimal | Real | Texto | Logico | Booleano | Caracter
SinoOpt   -> SiNo NL Bloque | epsilon
Args      -> Expr (, Expr)*
Expr      -> Expr O Expr | Expr Y Expr | Expr Comp Expr
          | Expr + Expr | Expr - Expr | Expr * Expr | Expr / Expr
          | - Expr | No Expr | (Expr) | NUMBER | STRING | CHAR
          | ID | Verdadero | Falso
```

Símbolo inicial: Programa. Los no terminales son las categorías a la izquierda de las producciones, incluyendo Expr, Sentencia y Tipo. Los terminales son palabras reservadas, constantes, identificadores, signos y saltos de línea. La producción anterior es una notación abreviada legible; la desambiguación de expresiones se hace por precedencia, con gramática detallada en README.md. Las restricciones de valor y de declaración se verifican semánticamente.

6. Programa demostrativo

```
Inicio
Definir edad Como Entero
Definir promedio Como Decimal
Definir nombre Como Texto
Definir aprobado Como Logico
Definir grupo Como Caracter
edad = 19
promedio = 89.5
nombre = "Ana"
aprobado = Verdadero
grupo = 'A'
Escribir "Estudiante:", nombre
Si aprobado Y promedio >= 60 Entonces
    Escribir "Aprobada"
SiNo
    Escribir "Pendiente"
FinSi
Fin
```

7. Las tres estructuras repetitivas

Para utiliza un contador numérico declarado previamente y admite un Paso opcional, que puede ser negativo. Mientras comprueba una condición antes de cada iteración. Repetir comprueba Hasta Que al final, de modo que el cuerpo se ejecuta al menos una vez. Todos los ciclos pueden anidarse y tienen un límite global de 10 000 instrucciones como medida preventiva.

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 3 Hacer
    Escribir i
FinPara
Fin

Mientras contador < 3 Hacer
    contador = contador + 1
FinMientras

Repetir
    intentos = intentos + 1
Hasta Que intentos == 3
```

8. Evidencia de las diez pruebas obligatorias

Los resultados siguientes provienen de ejecuciones reales del motor con Node.js. En cada caso se presenta código, entrada, resultado esperado, resultado obtenido y estado. Las evidencias en formato JSON/Markdown están incluidas en la carpeta tests/.

P01 - Aritmética - APROBADA

```
Inicio
Definir a Como Entero
a = 2 + 3 * 4
Escribir a
Fin
```

Entrada: Ninguna

Esperado: 14

Obtenido: 14

P02 - Aritmética - APROBADA

```
Inicio
Definir x Como Decimal
x = (10 - 4) / 2
Escribir x
Fin
```

Entrada: Ninguna

Esperado: 3

Obtenido: 3

P03 - Entrada y salida - APROBADA

```
Inicio
Definir nombre Como Texto
Leer nombre
Escribir nombre
Fin
```

Entrada: Ana

Esperado: Ana

Obtenido: Ana

P04 - Entrada y salida - APROBADA

```
Inicio
Definir edad Como Entero
Leer edad
Escribir edad + 1
Fin
```

Entrada: 20

Esperado: 21

Obtenido: 21

P05 - Condición verdadera - APROBADA

```
Inicio
Definir n Como Entero
n = 8
Si n > 5 Entonces
Escribir "SI"
SiNo
Escribir "NO"
FinSi
Fin
```

Entrada: Ninguna

Esperado: SI

Obtenido: SI

P06 - Condición falsa - APROBADA

```
Inicio
Definir n Como Entero
n = 2
Si n > 5 Entonces
Escribir "SI"
SiNo
Escribir "NO"
FinSi
Fin
```

Entrada: Ninguna

Esperado: NO

Obtenido: NO

P07 - Error léxico - APROBADA

```
Inicio
Escribir @
Fin
```

Entrada: Ninguna

Esperado: Error léxico (línea 2): Símbolo no permitido: @

Obtenido: Error léxico (línea 2): Símbolo no permitido: @

P08 - Error sintáctico - APROBADA

```
Inicio
Si 1 == 1
Escribir "SI"
FinSi
Fin
```

Entrada: Ninguna

Esperado: Error sintáctico (línea 2): Se esperaba 'Entonces' y se encontró '\n'

Obtenido: Error sintáctico (línea 2): Se esperaba 'Entonces' y se encontró '\n'

P09 - Error ejecución - APROBADA

```
Inicio
Definir x Como Decimal
x = 8 / 0
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): División entre cero

Obtenido: Error de ejecución (línea 3): División entre cero

P10 - Error ejecución - APROBADA

```
Inicio
Escribir x
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 2): Variable no declarada: x

Obtenido: Error de ejecución (línea 2): Variable no declarada: x

9. Pruebas adicionales de tipos de datos

Se registraron catorce casos adicionales para comprobar los nuevos tipos, alias, declaración múltiple, entrada interactiva, lógica, cortocircuito y diagnósticos de errores.

T01 - Entero y declaración múltiple - APROBADA

```
Inicio
Definir a, b Como Entero
a = 7
b = a * 3
Escribir a, b
Fin
```

Entrada: Ninguna

Esperado: 7 21

Obtenido: 7 21

T02 - Decimal y alias Real - APROBADA

```
Inicio
Definir x Como Real
x = (10 + 5) / 2
Escribir x
Fin
```

Entrada: Ninguna

Esperado: 7.5

Obtenido: 7.5

T03 - Texto y concatenación - APROBADA

```
Inicio
Definir nombre Como Texto
nombre = "Luisa"
Escribir "Hola, " + nombre
Fin
```

Entrada: Ninguna

Esperado: Hola, Luisa

Obtenido: Hola, Luisa

T04 - Lógico y alias Booleano - APROBADA

```
Inicio
Definir activo Como Booleano
activo = Verdadero
Escribir activo
Fin
```

Entrada: Ninguna

Esperado: Verdadero

Obtenido: Verdadero

T05 - Caracter y literal - APROBADA

```
Inicio
Definir letra Como Caracter
letra = 'Z'
Escribir letra
Fin
```

Entrada: Ninguna

Esperado: Z

Obtenido: Z

T06 - Operadores lógicos - APROBADA

```
Inicio
Definir a, b Como Logico
a = Verdadero
b = Falso
Si a Y No b Entonces
Escribir "CORRECTO"
SiNo
Escribir "INCORRECTO"
FinSi
Fin
```

Entrada: Ninguna

Esperado: CORRECTO

Obtenido: CORRECTO

T07 - Prioridad y cortocircuito - APROBADA

```
Inicio
Si Verdadero O (1 / 0 > 3) Entonces
Escribir "OK"
FinSi
Fin
```

Entrada: Ninguna

Esperado: OK

Obtenido: OK

T08 - Lectura Lógica - APROBADA

```
Inicio
Definir activo Como Logico
Leer activo
Escribir activo
Fin
```

Entrada: falso

Esperado: Falso

Obtenido: Falso

T09 - Lectura Caracter - APROBADA

```
Inicio
Definir inicial Como Caracter
Leer inicial
Escribir inicial
Fin
```

Entrada: Ñ

Esperado: Ñ

Obtenido: Ñ

T10 - Error tipo Entero - APROBADA

```
Inicio
Definir edad Como Entero
edad = 20.5
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): Se esperaba un número entero

Obtenido: Error de ejecución (línea 3): Se esperaba un número entero

T11 - Error literal Caracter - APROBADA

```
Inicio
Definir inicial Como Caracter
inicial = 'AB'
Fin
```

Entrada: Ninguna

Esperado: Error sintáctico (línea 3): Un Caracter debe contener exactamente un carácter, por ejemplo: 'A'

Obtenido: Error sintáctico (línea 3): Un Caracter debe contener exactamente un carácter, por ejemplo: 'A'

T12 - Error lectura Lógica - APROBADA

```
Inicio
Definir activo Como Logico
Leer activo
Fin
```

Entrada: sí

Esperado: Error de ejecución (línea 3): Escriba Verdadero o Falso para Logico

Obtenido: Error de ejecución (línea 3): Escriba Verdadero o Falso para Logico

T13 - Error al asignar Texto a Lógico - APROBADA

```
Inicio
Definir ok Como Logico
ok = "Verdadero"
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): Se esperaba Logico (Verdadero o Falso)

Obtenido: Error de ejecución (línea 3): Se esperaba Logico (Verdadero o Falso)

T14 - Comparación lógica inválida - APROBADA

```
Inicio
Si Verdadero > Falso Entonces
Escribir 1
FinSi
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 2): La comparación de orden requiere números o texto

Obtenido: Error de ejecución (línea 2): La comparación de orden requiere números o texto

10. Pruebas de repetidores

Veinte casos de prueba adicionales ejecutados con el motor real comprueban Para, Mientras, Repetir, anidamiento, validaciones y límites. Cada registro incluye código, entradas y resultado esperado y obtenido.

C01 - Para básico - APROBADA

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 4 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: 1
2
3
4

Obtenido: 1
2
3
4

C02 - Para con Paso descendente - APROBADA

```
Inicio
Definir i Como Entero
Para i = 5 Hasta 1 Paso -2 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: 5
3
1

Obtenido: 5
3
1

C03 - Para sin vueltas - APROBADA

```
Inicio
Definir i Como Entero
Para i = 5 Hasta 1 Hacer
Escribir i
FinPara
Escribir "Terminado"
Fin
```

Entrada: Ninguna

Esperado: Terminado

Obtenido: Terminado

C04 - Mientras con incremento - APROBADA

```
Inicio
Definir n Como Entero
n = 1
Mientras n <= 3 Hacer
Escribir n
n = n + 1
FinMientras
Fin
```

Entrada: Ninguna

Esperado: 1
2
3

Obtenido: 1
2
3

C05 - Mientras inicialmente falso - APROBADA

```
Inicio
Definir n Como Entero
n = 10
Mientras n < 5 Hacer
Escribir "Error"
FinMientras
Escribir "Listo"
Fin
```

Entrada: Ninguna

Esperado: Listo

Obtenido: Listo

C06 - Repetir hasta que - APROBADA

```
Inicio
Definir n Como Entero
n = 0
Repetir
n = n + 1
Escribir n
Hasta Que n >= 3
Fin
```

Entrada: Ninguna

Esperado: 1
2
3

Obtenido: 1
2
3

C07 - Repetir al menos una vez - APROBADA

```
Inicio
Definir n Como Entero
n = 4
Repetir
Escribir "Una vez"
Hasta Que n > 2
Fin
```

Entrada: Ninguna

Esperado: Una vez

Obtenido: Una vez

C08 - Bucles anidados - APROBADA

```
Inicio
Definir i, j Como Entero
Para i = 1 Hasta 2 Hacer
Para j = 1 Hasta 2 Hacer
Escribir i, j
FinPara
FinPara
Fin
```

Entrada: Ninguna

Esperado: 1 1
1 2
2 1
2 2

Obtenido: 1 1
1 2
2 1
2 2

C09 - Si dentro de Para - APROBADA

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 3 Hacer
Si i >= 2 Entonces
Escribir i
FinSi
FinPara
Fin
```

Entrada: Ninguna

Esperado: 2
3

Obtenido: 2
3

C10 - Entrada y Mientras - APROBADA

```
Inicio
Definir n Como Entero
Leer n
Mientras n < 3 Hacer
Escribir n
n = n + 1
FinMientras
Fin
```

Entrada: 1

Esperado: 1
2

Obtenido: 1
2

C11 - Paso cero - APROBADA

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 5 Paso 0 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): Paso no puede ser cero

Obtenido: Error de ejecución (línea 3): Paso no puede ser cero

C12 - Control no declarado - APROBADA

```
Inicio
Para i = 1 Hasta 2 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 2): Variable de control no declarada: i

Obtenido: Error de ejecución (línea 2): Variable de control no declarada: i

C13 - Control no numérico - APROBADA

```
Inicio
Definir i Como Texto
i = "n"
Para i = 1 Hasta 2 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 4): Para necesita una variable Entero o Decimal

Obtenido: Error de ejecución (línea 4): Para necesita una variable Entero o Decimal

C14 - Mientras con límite de seguridad - APROBADA

```
Inicio
Mientras Verdadero Hacer
Escribir "X"
FinMientras
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): Límite de instrucciones excedido

Obtenido: Error de ejecución (línea 3): Límite de instrucciones excedido

C15 - Para sin FinPara - APROBADA

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 3 Hacer
Escribir i
Fin
```

Entrada: Ninguna

Esperado: Error sintáctico (línea 5): Instrucción no válida: 'Fin'

Obtenido: Error sintáctico (línea 5): Instrucción no válida: 'Fin'

C16 - Mientras sin Hacer - APROBADA

```
Inicio
Definir x Como Entero
x = 1
Mientras x < 4
Escribir x
FinMientras
Fin
```

Entrada: Ninguna

Esperado: Error sintáctico (línea 4): Se esperaba 'Hacer' y se encontró '\n'

Obtenido: Error sintáctico (línea 4): Se esperaba 'Hacer' y se encontró '\n'

C17 - Repetir con condición no lógica - APROBADA

```
Inicio
Repetir
Escribir "Hola"
Hasta Que 3
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 2): Hasta Que requiere una condición Logico

Obtenido: Error de ejecución (línea 2): Hasta Que requiere una condición Logico

C18 - Para con paso incorrecto para Entero - APROBADA

```
Inicio
Definir i Como Entero
Para i = 1 Hasta 2 Paso 0.5 Hacer
Escribir i
FinPara
Fin
```

Entrada: Ninguna

Esperado: Error de ejecución (línea 3): Una variable Entero requiere inicio y paso enteros

Obtenido: Error de ejecución (línea 3): Una variable Entero requiere inicio y paso enteros

C19 - Para con decimal - APROBADA

```
Inicio
Definir x Como Decimal
Para x = 1.5 Hasta 2.5 Paso 0.5 Hacer
Escribir x
FinPara
Fin
```

Entrada: Ninguna

Esperado: 1.5
2
2.5

Obtenido: 1.5
2
2.5

C20 - Mientras con Logico - APROBADA

```
Inicio
Definir seguir Como Booleano
Definir n Como Entero
seguir = Verdadero
n = 0
Mientras seguir Hacer
n = n + 1
Si n == 2 Entonces
seguir = Falso
FinSi
FinMientras
Escribir n
Fin
```

Entrada: Ninguna

Esperado: 2

Obtenido: 2

11. Resumen de resultados y pruebas de interfaz

Pruebas obligatorias aprobadas: **10/10**. Tipos de datos: **14/14**. Ciclos: **20/20**. Total del intérprete: **44/44**. Además, 10/10 comprobaciones de interfaz aprobaron en Chromium, incluyendo apertura de pestañas, renombrado, ejecución, atajos y restauración de una sesión de prueba.

12. Repositorio GitHub

URL del repositorio: PENDIENTE DE PUBLICACIÓN. El equipo debe crear el repositorio en su propia cuenta e insertar aquí el enlace verdadero. No se afirma que el código ya esté publicado.

13. Conclusiones

El analizador léxico y el parser formalizan las reglas del lenguaje, mientras que el AST permite ejecutar solamente instrucciones reconocidas. El sistema comprueba declaraciones, tipos y expresiones, y ofrece diagnósticos por línea que facilitan el aprendizaje. El nuevo panel de tipos permite consultar y practicar Entero, Decimal, Texto, Logico y Caracter. Los ciclos Para, Mientras y Repetir amplían el aprendizaje de control de flujo y el editor multiarchivo favorece la comparación de algoritmos sin perder contenido. Las cuarenta y cuatro pruebas automatizadas y diez comprobaciones de interfaz, junto con las capturas, documentan las funciones entregadas.